

中山大学专业课《程序设计I》

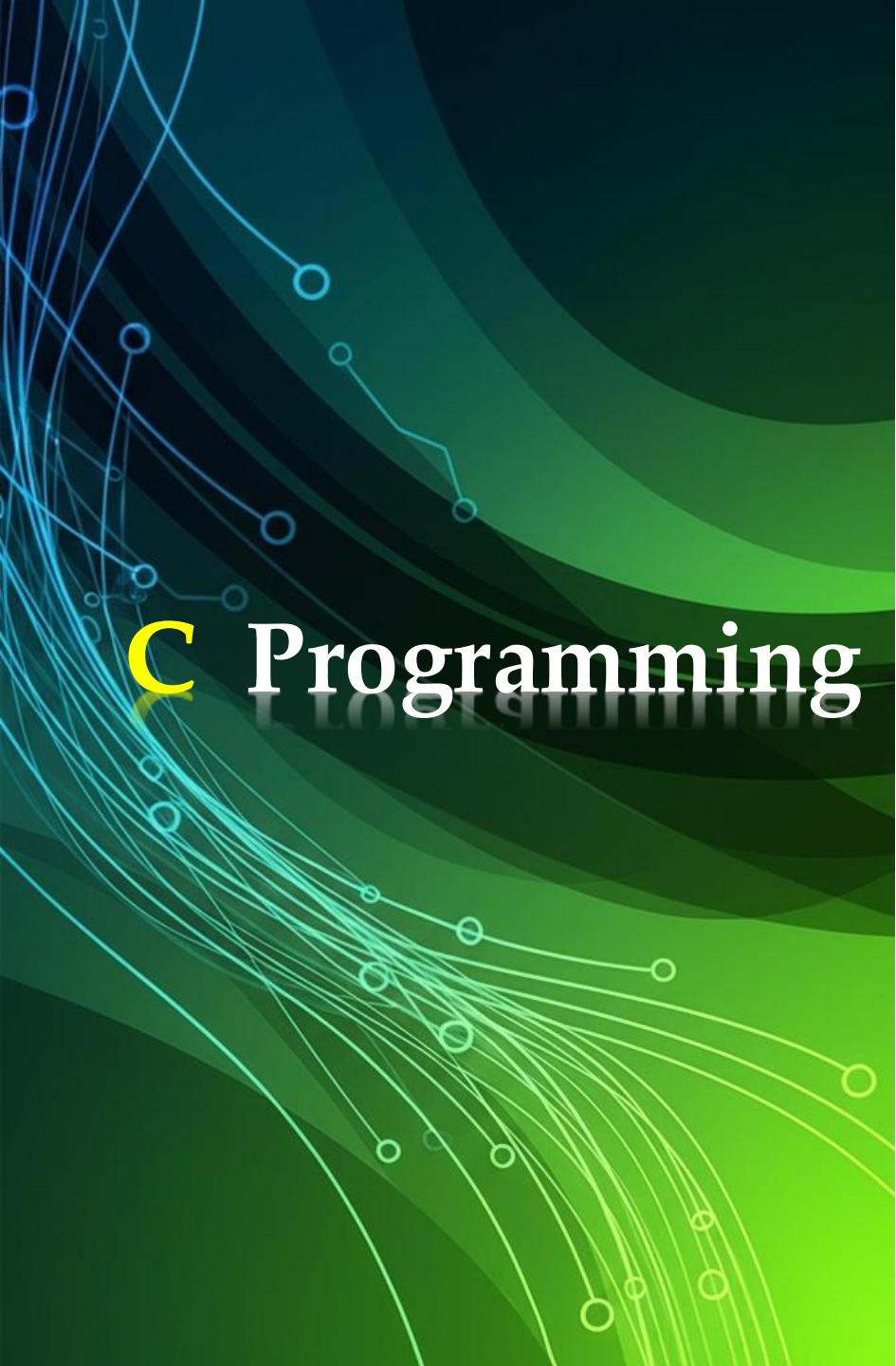
第七节 数组

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季



C Programming

大纲

1. 数组声明
2. 数组初始化
3. 数组调用与赋值
4. 二维数组与多维数组
5. 数组与函数

为什么要用数组?

数组可以储存一系列相同类型的数据

- 要向计算机输入全班100个学生期中考试的成绩

解决方法

用100个float型变量表示学生的成绩

- 烦琐，如果有1000名学生怎么办呢？
- 没有反映出这些数据间的内在联系，实际上这些数据是同一个班级、同一门课程的成绩，它们具有相同的属性。



一维数组声明

类型名 数组名[整型常量表达式¹]

`int a[10];`

数组元素类型，即数组中每一个元素均为int类型

数组名为a

数组包含10个int元素

数组元素的**下标从0开始**，用“`int a[10];`”定义数组，则最大下标值为9，第i个元素由`a[i-1]`表示，**不存在数组元素`a[10]`**

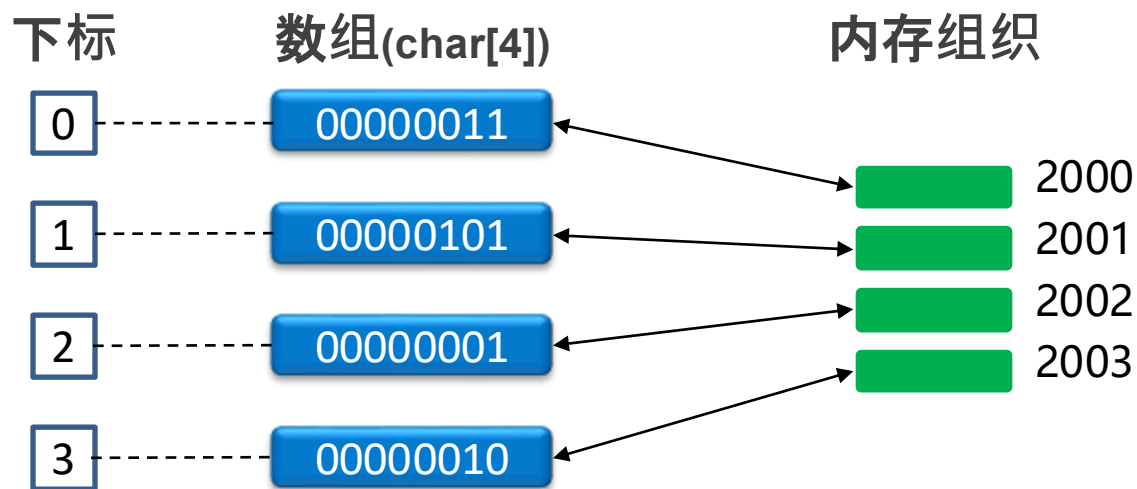
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9]

相当于声明了10个int变量，用0-9下标/索引来访问

¹ C99允许支持变长数组 (Variable Length Array, VLA)

数组：一种数据结构

- ❑ 数组 (array) 是由**相同类型**的元素 (element) 所组成的数据结构，元素对象的类型称为元素类型 (element type)
- ❑ 数组类型由**元素类型**与**数组长度**确定 (派生类型，如int[10]、int[100])
- ❑ 数组在内存中用一块**连续的内存**来存储，利用元素的索引/下标 (index) 可以计算出该元素对应的内存地址*



访问效率高!

一维数组的初始化

1. 对所有元素赋初值，例如 `int a[5]={0,1,2,3,4}` → 初始化列表
2. 对部分元素赋初值，例如 `int a[5]={0,1}` 只给前面2个元素赋初值，此时剩余元素会被自动赋0值
3. 不指定数组的长度，例如 `int a[]={0,1,2}`，它等价于 `int a[3]={0,1,2}`
4. C99 开始，开始支持可变长度数组 (Variable Length Array, VLA)，允许使用变量表示数组的长度（部分编译器可能不支持）*
5. 注意 `a[]={0,1,2}` 不是可变长度数组，a是由编译器根据初始值数量自动确定长度

一维数组的初始化

6. C99支持为指定数组元素初始化

```
/* 将数组第4个元素赋值为4 */  
int a1[5] = { [3]=4 }; // 其余为0
```

```
int a1[] = { [3]=4 }; // 不指定数组大小  
int a2[] = { [3]=4, 5, 6 }; // 多个值
```

* 可以使用sizeof()运算符计算数组所占的字节大小

一维数组的调用与赋值

运算符	运算符名	示例	描述
[]	数组下标	a [b]	访问数组 a 的第 b+1 个元素

1. 下标为**整型表达式**
2. 数组元素与一个简单变量的地位和作用相似

以下程序有什么问题？

【例7.1】对10个数组元素依次赋值为1,2,3,4,5,6,7,8,9,10，要求按逆序输出。*

```
#include <stdio.h>
int main(void)
{
    int i, a[];
    for(i=1; i<=10; i++)
        a[i] = i;
    for(i=10; i>=1; i--)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

需指定大小

数组越界

C语言为了程序执行更快，不检查越界行为

以下程序有什么问题？

【例7.1】对10个数组元素依次赋值为1,2,3,4,5,6,7,8,9,10，要求按逆序输出。 *

```
#include<stdio.h>
int main(void)
{
    int i,a[10],b[10];
    a = {0,1,2,3,4,5,6,7,8,9};
    b = a;
    for(i=9; i>=0; i--)
        printf("%d ",b[i]);
    printf("\n");
    return 0;
}
```

以下程序有什么问题？

【例6.1】对10个数组元素依次赋值为1,2,3,4,5,6,7,8,9,10，要求按逆序输出。

```
#include<stdio.h>
int main(void)
{
    int i,a[10],b[10];
    ✗ a = {0,1,2,3,4,5,6,7,8,9}; ----> 只能用于初始化
    ✗ b = a; -----> 不能把数组作为单元赋给另一个数组
    for(i=9; i>=0; i--)
        printf("%d ",a[i]);
    printf("\n");
    return 0;
}
```

数组输入

- ❑ 数组只能**逐个输入**元素，需通过**循环结合输入函数**依次读取
- ❑ 数组输入通常有以下三种情况*：
 - 输入第一个数代表要输入的个数 n ，后面输入 n 个数
 - 输入若干数（字符），最后一个数是特殊的数（字符），如 -1
 - 输入若干个数，最后是“文件结束”信号¹
 - C 语言的输入函数接收到信号后会返回 EOF（End Of File）
 - EOF 是一个整型符号常量（通常在 `stdio.h` 中定义），一般为 -1

¹ Windows：组合键 Ctrl + Z（通常后面需要按 Enter）；Linux/macOS：组合键 Ctrl + D

数组输入：程序样例

□ 数组只能逐个输入元素

```
//第一个数是数组的个数
#include<stdio.h>

#define MAX_LEN 100

int main() {
    int count, arr[MAX_LEN];

    scanf("%d",&count);
    for (int i=0; i<count; i++) {
        scanf("%d",&arr[i]);
    }

    for (int i=0; i<count; i++) printf("%d ",arr[i]);
    printf("\n");
    return 0;
}
```

```
//用特殊的数结束
...
    int count = 0;
    const int SPEC = -1;
    do {
        scanf("%d",&arr[count++]);
    } while (arr[count-1]!=SPEC);
    count--;    //?
...
```

```
//用 EOF 结束
#include<stdlib.h>

...
    int count = 0;
    while (scanf("%d",&arr[count++])!=EOF);
    count--;
...
```

数组常见错误总结

```
#define N 5

int main() {
    int arr1[]; // 没有声明元素数量

    int m=5;
    int arr2[m]; // C99可变长度数组/VLA（仅适用自动变量）

    int arr3[N];
    arr3 = {1,2,3,4,5}; // 数组标识符是值（地址），不能作为左值

    int arr4[N] = {1,2,3,4,5,6}; // 初始化超长度（warning）

    int arr5[N] = {1,2,3,4,5}, a[N];
    a = arr5; // a不是左值

    return 0;
}
```

常用数组处理方式

- ❑ 需要获得数组的长度: `#define LEN_ARRAY(a) (sizeof(a) / sizeof(a[0]))`
 - `sizeof(array)` 返回该数组定义的字节 (bytes) 数
- ❑ 类型隐式转换: `int a[] = {1, 2, 3.14, 4, 5};`
- ❑ 数组名是地址值 (右值), 它的值也是其**第一个元素的地址**, 即
 - `a == &(a[0])`, 是右值
 - `*a == *&(a[0]) == a[0]`, 是左值
- ❑ 数组元素赋零值*
 - `memset(arr, 0, sizeof(arr));`

练习

【例7.2】输入某市一个月降水数据，请编写程序计算该市一次降水的平均值。

输入样例：0,3.5,0,0,0,0,0,0,0,0,4.2,0,0,0,0,0,0,0,0,7.0,0,0,0,0,0,0,0,-1

```
/*average rain*/
#include<stdio.h>

int main() {
    double data[100];
    int n=0;

    /*读入一个数组，以 -1 作为结束*/
    do {
        scanf("%lf",&data[n]);
        n++;
    } while (data[n-1]!=-1);
    n--;
```

```
int rainedays = 0;
double rain_total = 0;
for (int i=0; i<n; i++) {
    if (data[i] > 1e-7) {
        rainedays++;
        rain_total += data[i];
    }
}

if (rainedays)
    printf("Rainedays %d, average rainfall\n",rainedays,rain_total/rainedays);
else
    printf("No rain.\n");
}
```

二维数组

现在有一个3*3的矩阵，需要保存在计算机当中

如果创建一个数组，应当是二维的，第一维代表第几行，第二维代表第几列

类型说明符 数组名[整型表达式][整型表达式]

int mat[3][3]

int mat[3][3]

mat[0] —— mat[0][0] mat[0][1] mat[0][2]

mat[1] —— mat[1][0] mat[1][1] mat[1][2]

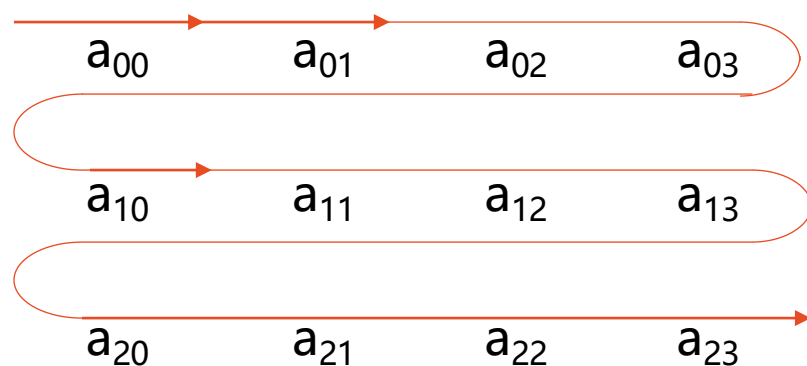
mat[2] —— mat[2][0] mat[2][1] mat[2][2]

mat[i][j]代表第i+1行第j+1列的元素

二维数组的储存

C语言中，二维数组在内存中是**按行存放**的

float a[3][4]



注意

- 用**矩阵形式**（如3行4列形式）表示二维数组，是**逻辑**上的概念，能形象地表示出行列关系。而在**内存**中，各元素是连续存放的，不是二维的，是**线性**的。

2000	a[0][0]	}
2004	a[0][1]	
2008	a[0][2]	
2012	a[0][3]	
2016	a[1][0]	}
2020	a[1][1]	
2024	a[1][2]	
2028	a[1][3]	
2032	a[2][0]	}
2036	a[2][1]	
2040	a[2][2]	
2044	a[2][3]	

二维数组的初始化

可以用“初始化列表”对二维数组初始化

1. 分行给二维数组赋初值（最清楚直观）

```
int a[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

2. 将所有数据写在一个花括号内，按内存中的顺序对各元素赋初值

```
int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

3. 若对全部元素都赋初值，则初始化时第1维长度可不指定，但第2维长度不能省

```
int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

≡

```
int a[][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

二维数组的初始化

赋值时只可省略第1维的长度!

4. 可以对部分元素赋初值 (按维度顺序考虑)

```
int a[3][4]={{1},{5},{9}};
```

①

```
int a[3][4]={{1},{0,6},{0,0,11}};
```

②

```
int a[3][4]={{1},{5,6}};
```

③

```
int a[3][4]={{1},{},{9}};
```

④

①

1	0	0	0
5	0	0	0
9	0	0	0

②

1	0	0	0
0	6	0	0
0	0	11	0

③

1	0	0	0
5	6	0	0
0	0	0	0

④

1	0	0	0
0	0	0	0
9	0	0	0

5. 也可以只对部分元素赋初值而省略第1维的长度, 但应分行赋初值

```
int a[][4]={{0,0,3},{},{0,10}};
```

引用二维数组元素

数组名[下标][下标]

- 下标可以是整型常量或整型表达式
- 数组元素可以出现在表达式中，也可以被赋值，如：b[1][2]=a[2][3]/2;

注意

- 在引用数组元素时，下标值应在已定义的数组大小的范围内。

```
int a[3][4]; //定义a为3×4的二维数组
```

```
a[3][4]=3; //不存在a[3][4]元素
```

//数组a可用的“行下标”的范围为0~2，“列下标”的范围为0~3

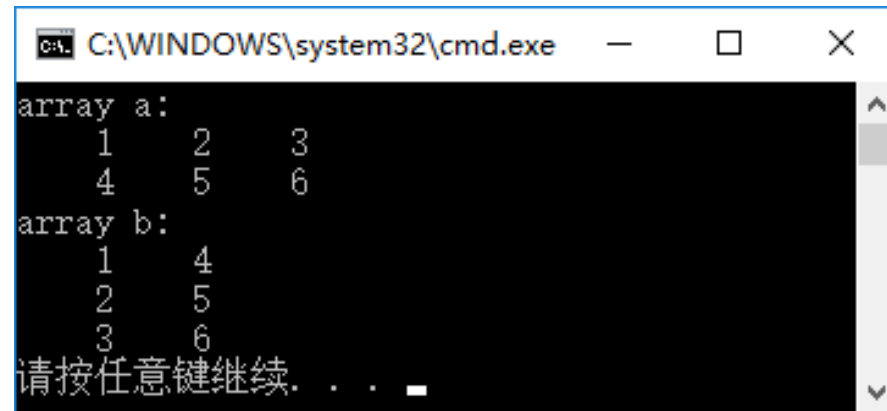


- 严格区分在**定义**数组时用的a[3][4]和**引用**元素时的a[3][4]的区别。前者用a[3][4]来定义数组的维数和各维的大小，后者a[3][4]中的3和4是数组元素的下标值，a[3][4]代表行序号为3、列序号为4的元素（行序号和列序号均从0起算）。

练习：矩阵转置

【例6.4】将以下二维数组行和列的元素互换，存到另一个二维数组中。

$$a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow b = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$



```
#include <stdio.h>
int main()
{
    int a[2][3]={{1,2,3},{4,5,6}};
    int b[3][2],i,j;
    printf("array a:\n");
    for(i=0;i<=1;i++)           //处理a数组中的一行中各元素
    {
        for (j=0;j<=2;j++)      //处理a数组中某一列中各元素
        {
            printf("%5d",a[i][j]); //输出a数组的一个元素
            b[j][i]=a[i][j]; //将a数组元素的值赋给b数组相应元素
        }
    }
```

```
        printf("\n");
    }
    printf("array b:\n");           //输出b数组各元素
    for(i=0;i<=2;i++)              //处理b数组中一行中各元素
    {
        for(j=0;j<=1;j++)          //处理b数组中一列中各元素
            printf("%5d",b[i][j]); //输出b数组的一个元素
        printf("\n");
    }
    return 0;
```

多维数组

```
float a[2][3][4]; // 定义三维数组a, 它有2页, 3行, 4列
```

可理解为

```
float[3][4] a[2]; // a是包含2元素的数组, 每个元素是一个3*4的数组
```

float a[2][3][4] 在内存中的排列顺序为:

a[0][0][0] → a[0][0][1] → a[0][0][2] → a[0][0][3] → a[0][1][0] → a[0][1][1] → a[0][1][2] →
a[0][1][3] → a[0][2][0] → a[0][2][1] → a[0][2][2] → a[0][2][3] → a[1][0][0] → a[1][0][1] →
a[1][0][2] → a[1][0][3] → a[1][1][0] → a[1][1][1] → a[1][1][2] → a[1][1][3] → a[1][2][0] →
a[1][2][1] → a[1][2][2] → a[1][2][3]

数组与函数

如何将一维数组作为参数传入某个函数？ 例如：计算两个向量的和

```
/*vector add*/
#include<stdio.h>
#define LEN_ARR(a) (sizeof(a) / sizeof(a[0]))
void vadd(int a[const], const int b[], const size_t sz);

int main(){
    int a[9] = {1,2,3,4,5,6,7,8,9};
    vadd(a,a,LEN_ARR(a));
    for (int i=0;i<LEN_ARR(a);i++)
        printf("%d,",a[i]);
    printf("\n");
}

void vadd(int a[const], const int b[], const size_t sz){
    for (int i=0;i<sz;i++) {
        a[i] += b[i];
    }
}
```

要点：

- ① 数组形参是一个变量，可保存数组的地址，用**不定长数组表示**，如 b[]；如果形参定义为 int b[10]，则编译忽略数组长度
- ② **const** 在函数申明中应用
 - int a[const] 表示形参 a 不可修改
 - const int b[] 表示b的元素不可修改
 - const **size_t** sz 表示sz不可修改
- ③ 函数中无法获得数组的长度，通常会将数组长度作为参数传给函数，类型是 size_t

数组与函数

如何将多维数组作为参数传入某个函数？

```
/*matrix add*/
#include<stdio.h>
... ..
void matrix_add(int a[][3], int b[][3], size_t m);

int main(){
    int a[3][3] = {1,2,3,4,5,6,7,8,9};
    matrix_add(a,a,LEN_ARR(a));
    for (int i=0;i<LEN_ARR(a);i++) {
        for (int j=0;j<LEN_ARR(a[i]);j++)
            printf("%4d,",a[i][j]);
        printf("\n");
    }
}

void matrix_add(int a[][3], int b[][3], size_t m){
    vadd(a,b,m*LEN_ARR(a[0]));
}
```

要点:

- ① 多维数组形参仅**第一维**用不定长数组表示, 如 b[][3]
- ② 如果形参定义为 int b[10][3], 则编译解释 int b[][3]
- ③ 多维数组形参除第一维外, 长度必须是常量

阅读左边程序与7_3_matrix_add.c
是否有哪里不懂？

谢谢！

邮箱: yuesh5@mail.sysu.edu.cn